#SKILL - 8 Spring Boot – JPQL & Query Methods Module Prerequisites:

- Basic Idea on Spring Boot and Spring Data JPA
- Knowledge of writing derived query methods

An e-commerce application needs a product search feature where users can view products by category, filter items within a price range, and sort products by price. The backend team must implement these operations using Spring Data JPA, including derived query methods and JPQL queries, to fetch data efficiently without writing long SQL statements.

To build a product search module that supports category-based search, price filtering, and sorting using Spring Data JPA.

Tasks:

1. Create a Product entity with the following fields: a. id b. name c. category d. price
2. Create a ProductRepository and add derived query methods:
    a. findByCategory(String category)
    b. findByPriceBetween(double min, double max)
3. Write JPQL queries using @Query for:
    a. Sorting products by price
    b. Fetching products above a price value
    c. Fetching products by category
4. Create REST controller endpoints to test query methods:
    a. /products/category/{category}
    b. /products/filter?min=&max=
    c. /products/sorted
    d. /products/expensive/{price}
5. Insert sample product records and test using Postman.
6. Verify output for different categories, ranges, and price values.
7. Push the Spring Boot project to GitHub.

Step -1 : Login to the Database(MySQL) and use the DATABASE hibernatedb created in Skill2.

- USE hibernatedb;

1. Open Command Prompt
2. Change the directory to where MySQL is installed

```
Command Prompt

Microsoft Windows [Version 10.0.26200.7171]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>cd C:\Program Files\MySQL\MySQL Server 8.0\bin
```

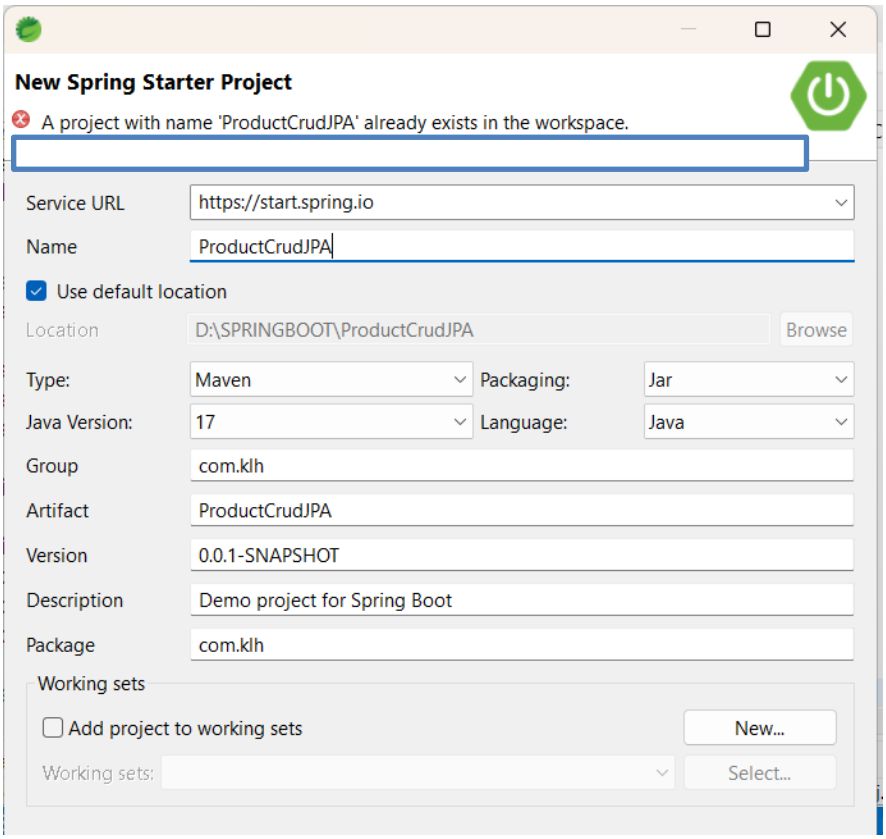
3. Enter the command
   - mysql -u root -p

```
C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql -u root -p
Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

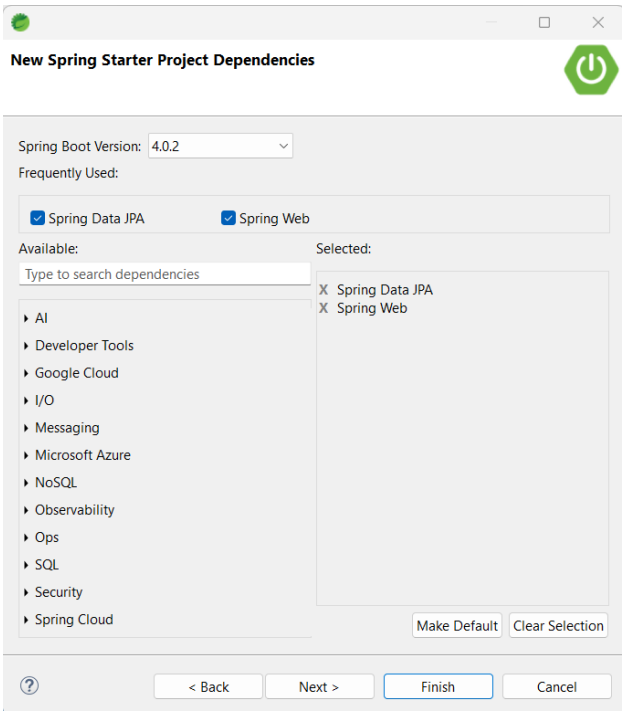
4. Change Database

```
mysql> USE hibernatedb;
Database changed
```

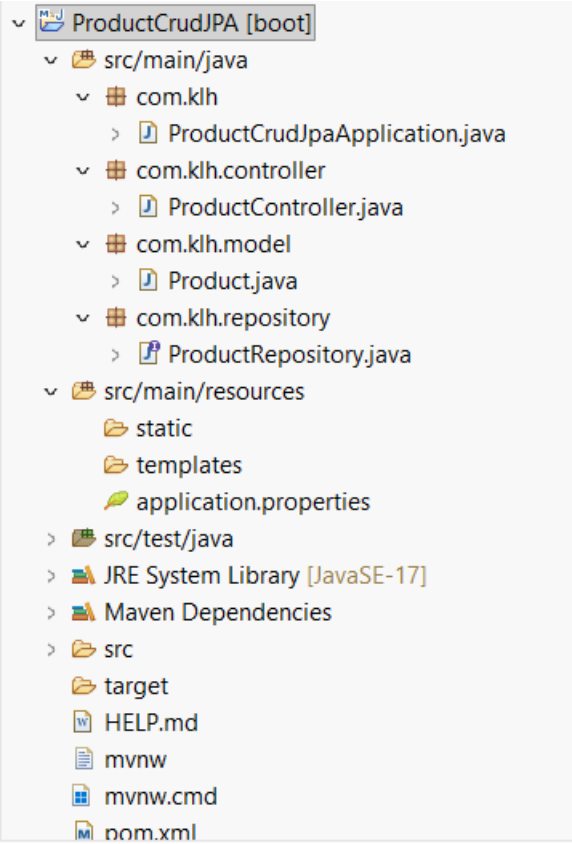
Step 2: Create a SpringStarter project in STS Tool:



Add the following dependencies and Click Finish



Step 3: Create Packages com.klh.model, repository and controller as shown below:



Step 4: Add the MySQL Dependency in **pom.xml** (Dependencies)

```
<!-- MySQL Driver -->
        <dependency>
        <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
        </dependency>
```

Step 5: Entity Class (Product.java).

```
package com.klh.model;
import jakarta.persistence.*;
@Entity
@Table(name="products")
public class Product {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;
   private String name;
   private String category;
   private double price;
   public Product() {}
   public Product(String name, String category, double price) {
      this.name = name;
      this.category = category;
      this.price = price;
   }
   // Getter and Setter methods
  public String getName() { return name; }
   public void setName(String name) { this.name = name; }
   public String getCategory() { return category; }
```

```
    public void setCategory(String category) { this.category = category; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
}
```

Step 6: Repository Class (ProductRepository .java)

```
package com.klh.repository;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import com.klh.model.Product;
public interface ProductRepository extends JpaRepository<Product, Long> {
   //Derived methods
    List<Product> findByCategory(String category);
    List<Product> findByPriceBetween(double min, double max);
    //JPQL Methods
   @Query("SELECT p FROM Product p ORDER BY p.price ASC")
    List<Product> getProductsSortedByPrice();
    @Query("SELECT p FROM Product p WHERE p.price > :price")
    List<Product> getExpensiveProducts(double price);
    @Query("SELECT p FROM Product p WHERE p.category = :category")
    List<Product> getProductsByCategoryJPQL(String category);
}
```

Step 7: Controller Class (ProductController.java)

```
package com.klh.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.web.bind.annotation.*;

import com.klh.model.Product;

import com.klh.repository.ProductRepository;

@RestController

@RequestMapping("/products")

public class ProductController {

   @Autowired

   private ProductRepository repo;

   @GetMapping("/category/{category}")

   public List<Product> getByCategory(@PathVariable String category){

      return repo.findByCategory(category);

   }

   @GetMapping("/filter")

   public List<Product> filterByPrice(@RequestParam double
min,@RequestParam double max){

      return repo.findByPriceBetween(min, max);

   }


   @GetMapping("/sorted")
```

```
    public List<Product> sortedProducts(){
        return repo.getProductsSortedByPrice();
    }
    @GetMapping("/expensive/{price}")
    public List<Product> expensiveProducts(@PathVariable double price){
        return repo.getExpensiveProducts(price);
    }
}
```

Step 8: Update application.properties (src->main->resources)

```
spring.application.name=ProductCrudJPA
# MySQL Connection
spring.datasource.url=jdbc:mysql://localhost:3306/hibernatedb
spring.datasource.username=root
spring.datasource.password=root
# Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
# Server
server.port=8080
```

Step 9: Run the project as SpringbootApp

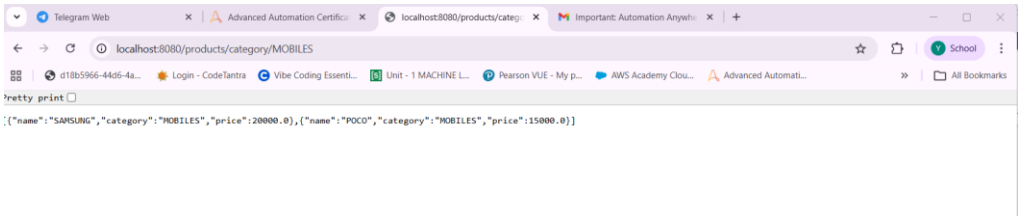
Step 10: Goto MySQL and insert values into the table

```
Default

mysql> INSERT INTO products(name, category, price) VALUES('SAMSUNG','MOBILES',20000);
Query OK, 1 row affected (0.02 sec)

mysql> INSERT INTO products(name, category, price) VALUES('SMARTWATCH','FASHION',4000);
Query OK, 1 row affected (0.04 sec)

mysql>  INSERT INTO products(name, category, price) VALUES('SPEAKERS','ACCESORIES',4000);
Query OK, 1 row affected (0.05 sec)

mysql>  INSERT INTO products(name, category, price) VALUES('ACER','LAPTOPS',60000);
Query OK, 1 row affected (0.05 sec)

mysql>  INSERT INTO products(name, category, price) VALUES('HP','LAPTOPS',67000);
Query OK, 1 row affected (0.05 sec)

mysql>  INSERT INTO products(name, category, price) VALUES('DELL','LAPTOPS',78000);
Query OK, 1 row affected (0.03 sec)

mysql> INSERT INTO products(name, category, price) VALUES('POCO','MOBILES',15000);
Query OK, 1 row affected (0.05 sec)

mysql>  INSERT INTO products(name, category, price) VALUES('WEBCAM','ACCESORIES',2000);
Query OK, 1 row affected (0.02 sec)

mysql> select * from products;
+----+-------------+------------+-------+
| id | category    | name       | price |
+----+-------------+------------+-------+
|  1 | Electronics | iPhone     | 80000 |
|  2 | Electronics | Laptop     | 60000 |
|  3 | Fashion     | Shoes      |  3000 |
|  4 | Fashion     | Watch      |  5000 |
|  5 | Furniture   | Chair      |  7000 |
|  6 | MOBILES     | SAMSUNG    | 20000 |
|  7 | FASHION     | SMARTWATCH |  4000 |
|  8 | ACCESORIES  | SPEAKERS   |  4000 |
|  9 | LAPTOPS     | ACER       | 60000 |
| 10 | LAPTOPS     | HP         | 67000 |
| 11 | LAPTOPS     | DELL       | 78000 |
| 12 | MOBILES     | POCO       | 15000 |
| 13 | ACCESORIES  | WEBCAM     |  2000 |
```

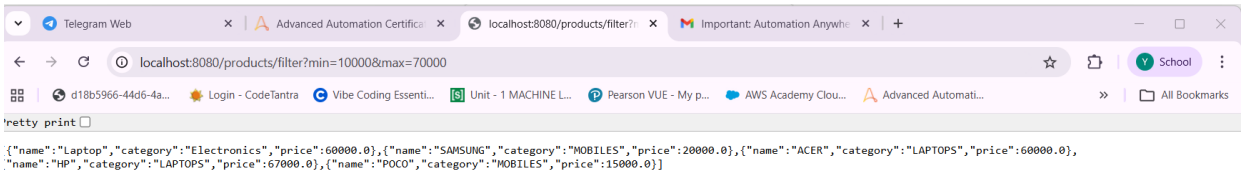
check the table in the database whether rows are inserted.

Step 11: Open the browser and test the REST controller endpoints:

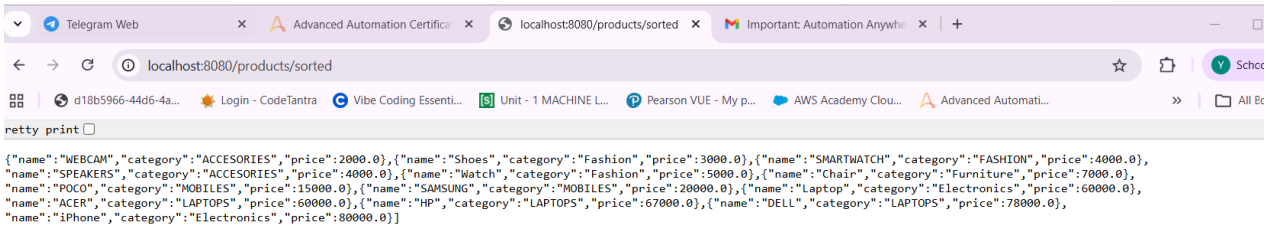
a. /products/category/MOBILES



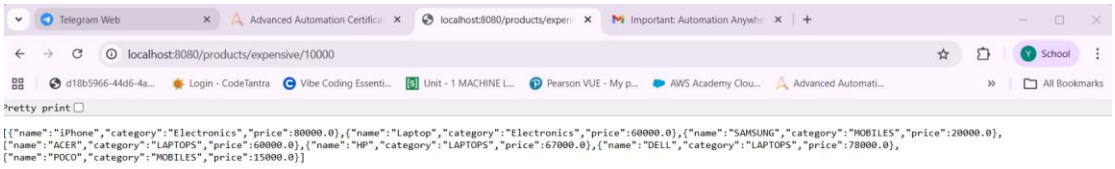
b. /products/filter?min=10000&max=70000



c. /products/sorted



d. /products/expensive/{price}



Step 12: Open POSTMAN App and test the same REST controller endpoints:

http://localhost:8080/products/sorted

Save | Share

GET http://localhost:8080/products/sorted Send

Docs | Params | Authorization | Headers (6) | Body | Scripts | Settings | Cookies

Body | Cookies | Headers (5) | Test Results | 200 OK • 20 ms • 884 B

JSON | Preview | Visualize

	name	category	price
0	WEBCAM	ACCESORIES	2000
1	Shoes	Fashion	3000
2	SMARTWATCH	FASHION	4000
3	SPEAKERS	ACCESORIES	4000
4	Watch	Fashion	5000
5	Chair	Furniture	7000
6	POCO	MOBILES	15000
7	SAMSUNG	MOBILES	20000
8	Laptop	Electronics	60000
9	ACER	LAPTOPS	60000
10	HP	LAPTOPS	67000
11	DELL	LAPTOPS	78000
12	iPhone	Electronics	80000

http://localhost:8080/products/category/FASHION

Save | Share

GET http://localhost:8080/products/category/FASHION Send

Docs | Params | Authorization | Headers (6) | Body | Scripts | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | Cookies | Headers (5) | Test Results | 200 OK • 17 ms • 329 B

JSON | Preview | Visualize

	name	category	price
0	Shoes	Fashion	3000
1	Watch	Fashion	5000
2	SMARTWATCH	FASHION	4000

http://localhost:8080/products/filter

Save | Share

GET http://localhost:8080/products/filter?min=10000&max=70000 Send

Docs | Params | Authorization | Headers (6) | Body | Scripts | Settings | Cookies

☑	min	10000	
☑	max	70000	
	Key	Value	Description

Body | Cookies | Headers (5) | Test Results

200 OK • 15 ms • 437 B

{} JSON | Preview | Visualize

	name	category	price
0	Laptop	Electronics	60000
1	SAMSUNG	MOBILES	20000
2	ACER	LAPTOPS	60000
3	HP	LAPTOPS	67000
4	POCO	MOBILES	15000

http://localhost:8080/products/expensive/10000

Save | Share

GET http://localhost:8080/products/expensive/10000 Send

Docs | Params | Authorization | Headers (6) | Body | Scripts | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | Cookies | Headers (5) | Test Results

200 OK • 19 ms • 549 B

{} JSON | Preview | Visualize

	name	category	price
0	iPhone	Electronics	80000
1	Laptop	Electronics	60000
2	SAMSUNG	MOBILES	20000
3	ACER	LAPTOPS	60000
4	HP	LAPTOPS	67000
5	DELL	LAPTOPS	78000
6	POCO	MOBILES	15000